

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление 02.03.01 Математика и компьютерные науки

Lessons and Exams Timetable Bot

Архитектура

Гладков И.А., Саржан А.А., Смирнов С.Р., Фролов К.А.

Санкт-Петербург, 2026

Содержание

1	Компонентная диаграмма	5
1.1	Сервер	5
1.2	Пользователь (User)	5
1.3	Администратор (Admin)	6
1.4	API layer	6
1.4.1	API (Telegram message handlers)	6
1.4.2	API (Healthcheck)	6
1.4.3	API (Admin level)	7
1.5	Модули системы	7
1.5.1	MessageHandlerModule	7
1.5.2	UserModule	8
1.5.3	TimetableModule	8
1.5.4	TimetableImportModule	8
1.5.5	TaskModule	9
1.5.6	ReminderModule	9
1.5.7	NotificationModule	9
1.5.8	AdvisorModule	10
1.5.9	CalendarSyncModule	10
1.5.10	StudySessionModule	10
1.5.11	AdminAuthModule	11
1.5.12	ListUsersModule	11
1.5.13	HealthCheckModule	11
1.6	Репозитории	11
1.6.1	UserRepository	12
1.6.2	AdminRepository	12
1.6.3	TimetableEventRepository	12
1.6.4	StudyTaskRepository	12
1.6.5	ReminderRepository	13
1.6.6	StudySessionRepository	13
1.6.7	CalendarSyncRepository	13
1.6.8	ImportJobRepository	13
1.7	External API adapters	14
1.7.1	API (MongoDB)	14
1.7.2	API (RabbitMQ)	14
1.7.3	API (LLM)	14
1.7.4	API (Calendar)	15

1.7.5	API (Institution Timetable)	15
1.8	MongoDB	15
1.9	RabbitMQ	16
1.10	LLM API	16
1.11	Google/Yandex Calendar API	16
1.12	Institution Timetable API	16
1.13	Потоки данных на диаграмме	17

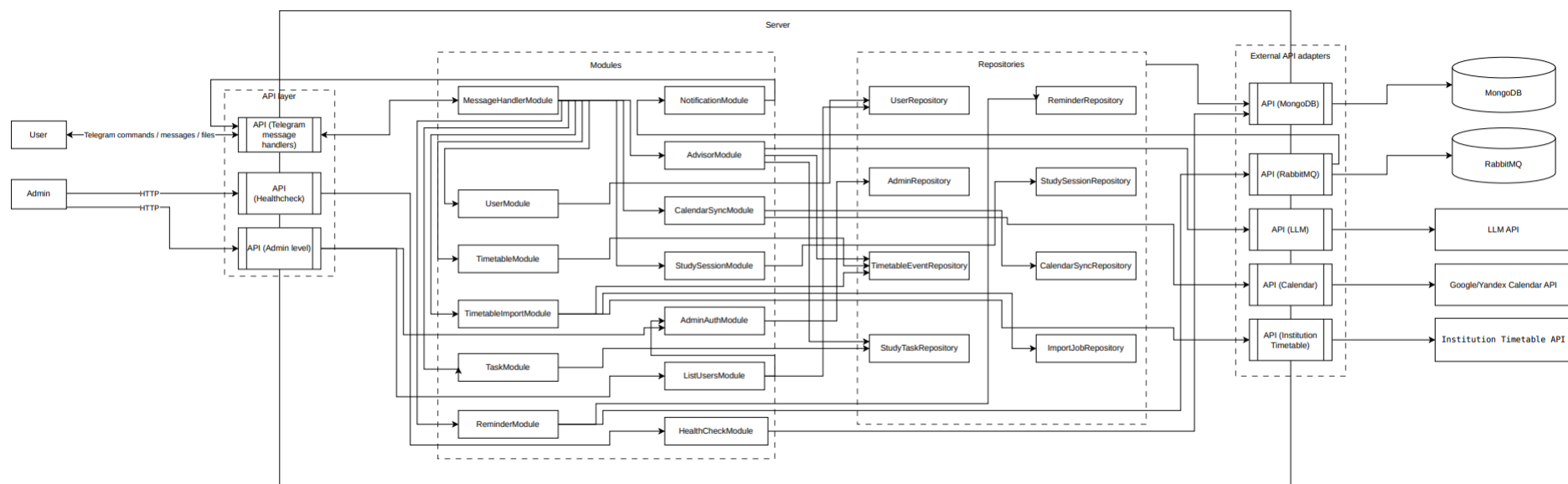


Рис. 1. Архитектура

1 Компонентная диаграмма

На компонентной диаграмме представлена архитектура Telegram-бота Lessons and Exams Timetable Bot. Система построена как серверное приложение на Java и Spring WebFlux. Внутри сервера выделены следующие логические слои: входной API layer, бизнес-модули, репозитории и адаптеры для взаимодействия с внешними системами.

Основной пользовательский сценарий начинается с отправки команды или файла через Telegram. Запрос поступает в слой API, затем передаётся в `MessageHandlerModule`, который определяет тип команды и направляет её в соответствующий бизнес-модуль. Бизнес-модули работают с данными через репозитории и внешние API-адаптеры. Данные пользователей, расписаний, задач, напоминаний и учебных сессий сохраняются в MongoDB. Для асинхронной обработки напоминаний используется RabbitMQ. Диаграмма представлена на рисунке 2.

1.1 Сервер

Роль: центральный компонент системы.

Функционал:

- принимает пользовательские команды, сообщения и файлы через Telegram;
- принимает HTTP-запросы администратора;
- обрабатывает служебный endpoint `/healthcheck`;
- координирует взаимодействие между API layer, бизнес-модулями, репозиториями и внешними API;
- обеспечивает работу с MongoDB, RabbitMQ, LLM API, календарными API и внешним API расписания учебного заведения.

1.2 Пользователь (User)

Роль: основной конечный пользователь Telegram-бота.

Функционал:

- взаимодействует с системой через Telegram;
- отправляет команды для просмотра расписания, управления задачами, настройки напоминаний и получения LLM-рекомендаций;
- может загружать файл расписания в формате CSV или iCal;
- идентифицируется автоматически по Telegram `chatId` или `telegramId`;
- может выполнить проверку состояния системы через endpoint `/healthcheck`, если такой endpoint доступен публично.

1.3 Администратор (Admin)

Роль: пользователь с расширенными правами.

Функционал:

- взаимодействует с системой через HTTP API;
- получает API-ключ или Bearer-токен через endpoint `/auth`;
- обращается к административному endpoint `/users`;
- может получать список пользователей системы;
- может выполнять проверку состояния сервера через endpoint `/healthcheck`.

1.4 API layer

Роль: входной слой приложения.

Функционал:

- принимает входящие запросы от пользователей и администратора;
- преобразует внешние запросы в формат, удобный для бизнес-модулей;
- передаёт команды, данные пользователя, файлы и HTTP-запросы в соответствующие модули.

1.4.1 API (Telegram message handlers)

Роль: обработка входящих Telegram-сообщений и команд.

Функционал:

- принимает команды пользователя, например `/start`, `/today`, `/week`, `/add_task`, `/advisor`;
- принимает файлы расписания для импорта;
- извлекает из сообщения `chatId`, `telegramId`, текст команды, `callback data` и `fileId`;
- передаёт распознанную команду в `MessageHandlerModule`;
- отправляет пользователю ответы, уведомления и сообщения об ошибках.

1.4.2 API (Healthcheck)

Роль: служебный endpoint для проверки состояния системы.

Функционал:

- обрабатывает запрос `GET /healthcheck`;
- не требует административной авторизации;

- передаёт запрос в `HealthCheckModule`;
- возвращает состояние сервера, состояние подключения к MongoDB и информацию об авторах проекта.

1.4.3 API (Admin level)

Роль: входной HTTP API для административных функций.

Функционал:

- обрабатывает запрос `/auth` для получения токена администратора;
- обрабатывает запрос `/users` для получения списка пользователей;
- передаёт логин, пароль и Bearer-токен в `AdminAuthModule`;
- передаёт авторизованные запросы в `ListUsersModule`.

1.5 Модули системы

1.5.1 MessageHandlerModule

Роль: центральный маршрутизатор пользовательских команд.

Функционал:

- получает распознанную Telegram-команду из API (`Telegram message handlers`);
- определяет тип команды и выбирает нужный бизнес-модуль;
- направляет команды расписания в `TimetableModule`;
- направляет команды импорта в `TimetableImportModule`;
- направляет команды управления задачами в `TaskModule`;
- направляет команды напоминаний в `ReminderModule`;
- направляет запросы к LLM-советнику в `AdvisorModule`;
- направляет команды синхронизации календаря в `CalendarSyncModule`;
- направляет команды учёта учебного времени в `StudySessionModule`;
- получает результат выполнения операции и формирует ответ пользователю.

1.5.2 UserModule

Роль: управление пользователями Telegram-бота.

Функционал:

- обрабатывает команду `/start`;
- идентифицирует пользователя по Telegram `chatId` или `telegramId`;
- создаёт нового пользователя при первом обращении к боту;
- хранит настройки пользователя: имя, `username`, часовой пояс, настройки напоминаний;
- обращается к `UserRepository` для чтения и сохранения пользовательских данных.

1.5.3 TimetableModule

Роль: управление расписанием занятий и экзаменов.

Функционал:

- выводит расписание на сегодня, завтра и текущую неделю;
- добавляет занятия и экзамены вручную;
- обновляет данные существующих занятий и экзаменов;
- удаляет события расписания;
- группирует события по дате и сортирует их по времени начала;
- обращается к `TimetableEventRepository` для работы с событиями расписания.

1.5.4 TimetableImportModule

Роль: импорт расписания из файлов и внешних источников.

Функционал:

- принимает CSV- или iCal-файл, загруженный пользователем;
- выполняет проверку формата файла;
- разбирает строки расписания и преобразует их в события системы;
- валидирует дату, время, тип события, предмет и обязательные поля;
- может получать расписание из внешнего API учебного заведения через API (`Institution Timetable`);
- сохраняет корректные занятия и экзамены через `TimetableEventRepository`;
- сохраняет результат импорта и ошибки через `ImportJobRepository`.

1.5.5 TaskModule

Роль: управление учебными задачами пользователя.

Функционал:

- добавляет учебные задачи, дедлайны, упражнения, лабораторные и курсовые работы;
- связывает задачи с учебными предметами;
- выводит задачи на сегодня, завтра и неделю;
- выводит задачи по выбранному предмету;
- изменяет статус задачи на выполненную;
- обращается к `StudyTaskRepository` для сохранения и получения задач.

1.5.6 ReminderModule

Роль: управление напоминаниями.

Функционал:

- хранит и изменяет настройки напоминаний пользователя;
- создаёт напоминания о занятиях, экзаменах и дедлайнах;
- определяет время отправки уведомлений;
- обращается к `ReminderRepository` для хранения настроек и задач напоминаний;
- публикует задачи на отставку уведомлений через API (`RabbitMQ`).

1.5.7 NotificationModule

Роль: отправка уведомлений пользователю.

Функционал:

- получает задачи уведомлений из `RabbitMQ` через API (`RabbitMQ`);
- формирует текст уведомления для пользователя;
- отправляет напоминания о занятиях, экзаменах и дедлайнах через API (`Telegram message handlers`);
- обрабатывает ошибки отправки сообщений.

1.5.8 AdvisorModule

Роль: LLM-советник по учебному планированию.

Функционал:

- обрабатывает команды `/priority`, `/exam_plan` и `/ask_advisor`;
- собирает контекст пользователя: расписание, экзамены, задачи, дедлайны и приоритеты;
- обращается к `TimetableEventRepository` и `StudyTaskRepository` для получения учебного контекста;
- формирует prompt для внешней LLM;
- отправляет запрос во внешний LLM API через API (LLM);
- возвращает пользователю учебные приоритеты, план подготовки к экзамену или ответ на свободный вопрос.

1.5.9 CalendarSyncModule

Роль: синхронизация расписания и задач с внешним календарём.

Функционал:

- обрабатывает команды синхронизации календаря;
- передаёт занятия, экзамены и дедлайны во внешний календарь;
- работает с Google Calendar API или Yandex Calendar API через API (Calendar);
- сохраняет связь между локальными событиями и внешними событиями календаря через `CalendarSyncRepository`;
- обрабатывает ошибки синхронизации.

1.5.10 StudySessionModule

Роль: учёт учебного времени пользователя.

Функционал:

- обрабатывает команды `/study_start`, `/study_stop` и `/study_stats`;
- фиксирует начало и окончание учебной сессии;
- связывает учебную сессию с предметом;
- рассчитывает длительность учебной сессии;
- предоставляет статистику по предметам и периодам;
- обращается к `StudySessionRepository` для хранения истории занятий.

1.5.11 AdminAuthModule

Роль: аутентификация и авторизация администратора.

Функционал:

- обрабатывает запросы к endpoint `/auth`;
- проверяет логин и пароль администратора;
- обращается к `AdminRepository` для получения данных администратора;
- генерирует API-ключ или Beager-токен при успешной аутентификации;
- проверяет токен при обращении к защищённым административным endpoint-ам.

1.5.12 ListUsersModule

Роль: получение списка пользователей для администратора.

Функционал:

- обрабатывает административный запрос `/users`;
- получает список пользователей через `UserRepository`;
- возвращает администратору Telegram ID, username и другую служебную информацию о пользователях;
- использует `AdminAuthModule` для проверки прав доступа.

1.5.13 HealthCheckModule

Роль: проверка состояния системы.

Функционал:

- обрабатывает запросы от API (`Healthcheck`);
- проверяет состояние серверного приложения;
- проверяет доступность MongoDB;
- при необходимости проверяет доступность RabbitMQ;
- возвращает статус системы и список авторов проекта.

1.6 Репозитории

Роль: слой доступа к данным MongoDB.

Функционал:

- предоставляет бизнес-модулям операции чтения, записи, обновления и удаления данных;

- инкапсулирует работу с коллекциями MongoDB;
- передаёт запросы к базе данных через API (MongoDB).

1.6.1 UserRepository

Роль: хранение данных пользователей.

Функционал:

- сохраняет Telegram ID, chatId, username и настройки пользователя;
- предоставляет поиск пользователя по Telegram ID;
- используется модулями `UserModule` и `ListUsersModule`.

1.6.2 AdminRepository

Роль: хранение данных администраторов.

Функционал:

- хранит логин администратора и хеш пароля;
- используется модулем `AdminAuthModule`;
- не хранит пароль администратора в открытом виде.

1.6.3 TimetableEventRepository

Роль: хранение занятий и экзаменов.

Функционал:

- хранит события расписания пользователя;
- различает типы событий: занятие и экзамен;
- предоставляет поиск событий на день, неделю и по предмету;
- используется модулями `TimetableModule`, `TimetableImportModule` и `AdvisorModule`.

1.6.4 StudyTaskRepository

Роль: хранение учебных задач.

Функционал:

- хранит задачи, дедлайны, приоритеты и статусы выполнения;
- предоставляет поиск задач по предмету и периоду;
- используется модулями `TaskModule` и `AdvisorModule`.

1.6.5 `ReminderRepository`

Роль: хранение настроек и задач напоминаний.

Функционал:

- хранит настройки времени напоминаний;
- хранит информацию о запланированных уведомлениях;
- используется модулем `ReminderModule`.

1.6.6 `StudySessionRepository`

Роль: хранение учебных сессий.

Функционал:

- хранит начало, окончание и длительность учебной сессии;
- связывает сессию с пользователем и предметом;
- используется модулем `StudySessionModule`.

1.6.7 `CalendarSyncRepository`

Роль: хранение информации о синхронизации с календарём.

Функционал:

- хранит связь между локальным событием и внешним событием календаря;
- хранит статус последней синхронизации;
- используется модулем `CalendarSyncModule`.

1.6.8 `ImportJobRepository`

Роль: хранение истории импорта расписания.

Функционал:

- сохраняет статус импорта расписания;
- хранит количество успешно добавленных и пропущенных записей;
- хранит ошибки импорта;
- используется модулем `TimetableImportModule`.

1.7 External API adapters

Роль: адаптеры для взаимодействия с внешними системами.

Функционал:

- скрывают детали конкретных внешних API от бизнес-модулей;
- выполняют сериализацию и десериализацию запросов;
- обрабатывают ошибки внешних сервисов;
- возвращают бизнес-модулям данные в едином формате.

1.7.1 API (MongoDB)

Роль: адаптер доступа к MongoDB.

Функционал:

- обеспечивает подключение к MongoDB;
- выполняет CRUD-операции с документами;
- используется всеми репозиториями системы.

1.7.2 API (RabbitMQ)

Роль: адаптер взаимодействия с RabbitMQ.

Функционал:

- публикует сообщения о запланированных уведомлениях;
- читает сообщения из очередей;
- передаёт задачи уведомлений в `NotificationModule`;
- обеспечивает асинхронную обработку напоминаний.

1.7.3 API (LLM)

Роль: адаптер взаимодействия с внешним LLM API.

Функционал:

- принимает prompt от `AdvisorModule`;
- отправляет запрос во внешний LLM API;
- получает ответ модели;
- возвращает учебные рекомендации, план подготовки или ответ на свободный вопрос пользователя.

1.7.4 API (Calendar)

Роль: адаптер взаимодействия с внешними календарями.

Функционал:

- создаёт события во внешнем календаре;
- обновляет и удаляет события при изменении локального расписания;
- получает внешние идентификаторы календарных событий;
- используется модулем `CalendarSyncModule`.

1.7.5 API (Institution Timetable)

Роль: адаптер взаимодействия с API расписания учебного заведения.

Функционал:

- отправляет запросы во внешний API расписания;
- получает список занятий и экзаменов;
- передаёт полученные данные в `TimetableImportModule`;
- используется как один из источников импорта расписания.

1.8 MongoDB

Роль: основная база данных системы.

Функционал:

- хранит пользователей;
- хранит занятия и экзамены;
- хранит учебные задачи и дедлайны;
- хранит настройки напоминаний;
- хранит учебные сессии;
- хранит данные синхронизации с внешними календарями;
- хранит историю импортов расписания;
- хранит данные администраторов.

1.9 RabbitMQ

Роль: брокер сообщений для асинхронной обработки задач.

Функционал:

- принимает сообщения о необходимости отправки напоминаний;
- хранит задачи уведомлений до момента обработки;
- передаёт сообщения в `NotificationModule`;
- повышает надёжность отправки уведомлений и отделяет логику планирования от логики отправки.

1.10 LLM API

Роль: внешний сервис генерации учебных рекомендаций.

Функционал:

- принимает prompt с учебным контекстом пользователя;
- генерирует рекомендации по приоритетам;
- генерирует план подготовки к экзамену;
- отвечает на свободные вопросы пользователя о планировании учёбы.

1.11 Google/Yandex Calendar API

Роль: внешний календарный сервис.

Функционал:

- принимает запросы на создание календарных событий;
- обновляет события при изменении расписания или дедлайнов;
- удаляет события при отключении синхронизации;
- возвращает внешние идентификаторы созданных событий.

1.12 Institution Timetable API

Роль: внешний источник расписания учебного заведения.

Функционал:

- предоставляет расписание занятий и экзаменов;
- может использовать номер группы, направление или другой идентификатор учебной группы;
- возвращает данные, которые затем преобразуются в события локального расписания пользователя.

1.13 Потоки данных на диаграмме

Пользователь отправляет Telegram-команды, сообщения и файлы в API (Telegram message handlers). Далее запрос передаётся в MessageHandlerModule, который маршрутизирует его в нужный модуль.

При работе с расписанием MessageHandlerModule вызывает TimetableModule. Модуль получает или изменяет занятия и экзамены через TimetableEventRepository, который выполняет операции с MongoDB через API (MongoDB).

При импорте расписания TimetableImportModule получает файл CSV/iCal или данные из Institution Timetable API. После валидации данные сохраняются через TimetableEventRepository. Результаты импорта и ошибки сохраняются через ImportJobRepository.

При работе с задачами TaskModule сохраняет и получает задачи через StudyTaskRepository. Задачи могут использоваться не только для вывода списков, но и как часть контекста для AdvisorModule.

При настройке напоминаний ReminderModule сохраняет параметры через ReminderRepository и публикует задачу уведомления через API (RabbitMQ). Затем NotificationModule получает задачу уведомления и отправляет сообщение пользователю через Telegram.

При обращении к LLM-советнику AdvisorModule собирает расписание, экзамены, задачи и дедлайны пользователя, формирует prompt и отправляет его через API (LLM) во внешний LLM API. Ответ модели возвращается пользователю через Telegram.

При синхронизации календаря CalendarSyncModule передаёт занятия, экзамены и дедлайны через API (Calendar) во внешний Google/Yandex Calendar API. Связь между локальными и внешними событиями сохраняется через CalendarSyncRepository.

Администратор обращается к API (Admin level). Для получения доступа используется endpoint /auth, а для получения списка пользователей – endpoint /users. Проверка данных администратора выполняется в AdminAuthModule, а список пользователей получается через ListUsersModule и UserRepository.

Endpoint /healthcheck обрабатывается через API (Healthcheck) и HealthCheckModule. Он возвращает состояние сервера, состояние подключения к MongoDB и служебную информацию о проекте.

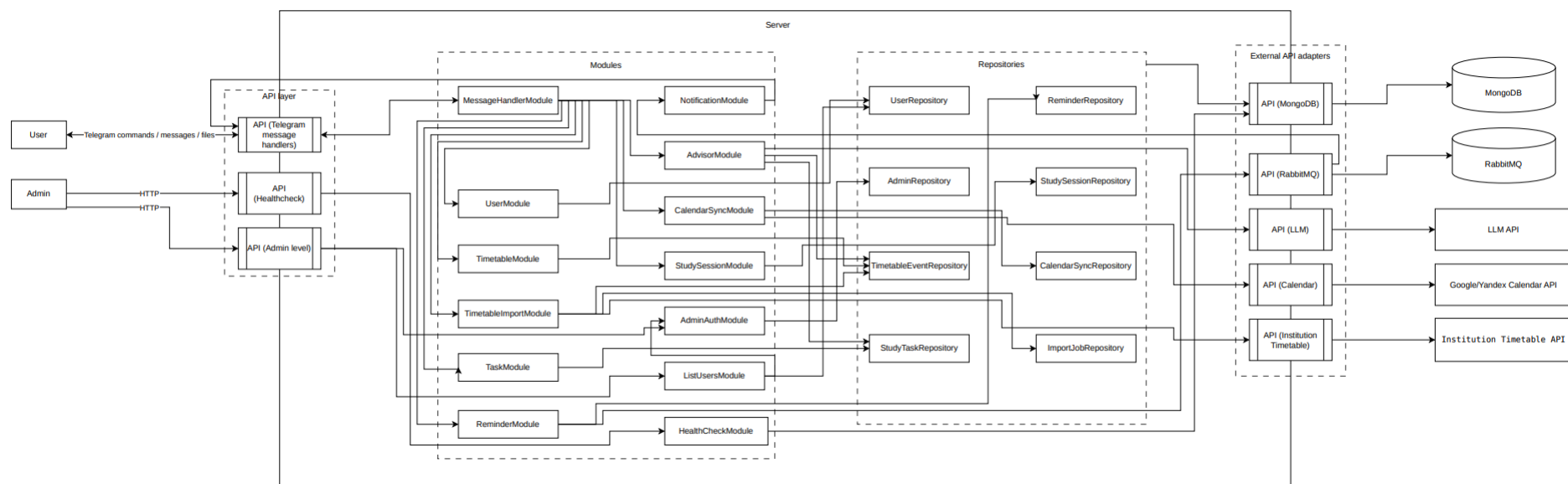


Рис. 2. Архитектура